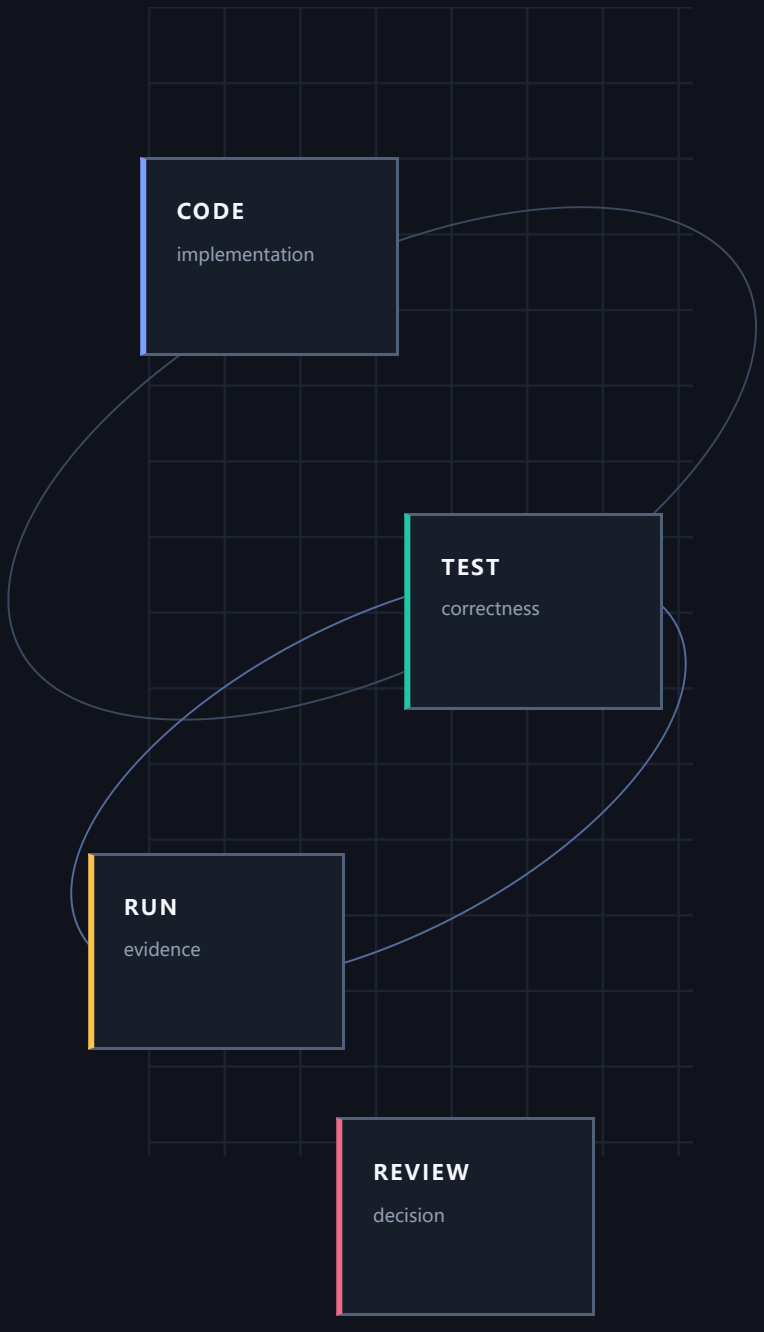
01 / SCOPE

Evidence first engineering.

A selected technical portfolio for coding evaluation, agent evaluation, STEM reasoning, software QA and code review.

The work is presented as an engineering record: source, tests, build, runtime evidence and explicit limits.



SOURCE / BUILD / RUNTIME / ACCEPTANCE

7 selected projects	**6** local systems re-run in copies	**3** public Python suites re-run	**0** credentials or private sources used

WHAT THE WORK SHOWS

Evaluation is a workflow, not a slogan.

Across the selected projects, implementation is paired with inspection. The recurring pattern is to define the contract, make the result observable, exercise failure paths and keep the final decision traceable.

01 Requirement -> 02 Specification -> 03 Implement -> 04 Review -> 05 Build -> 06 Test -> 07 Regression -> 08 Accept

CAPABILITY	PROJECT EVIDENCE	WHAT IS VERIFIED
Agent evaluation long-horizon decisions	**agentic-gpu-optimizer**	Isolated patches, correctness gate, benchmark gate, ACCEPT / REJECT / INCONCLUSIVE.
STEM reasoning algorithms and systems	**gpu-kernel-performance-lab**	Transpose, reduction and GEMM kernels; non-regular dimensions; CPU fallback kept separate from CUDA claims.
AI safety trust and evidence	**RAG poisoning detection**	Source risk, conflict graph, evidence factors, red-team samples, quarantine and self-purification.
Software QA bug finding and proof	**02-qa-sentinel**	HTTP, console, links, images, forms, SEO and visible overflow across desktop, tablet and mobile.
Security engineering failure boundaries	**04-webhook-observatory**	Timing-safe HMAC verification, schema contracts, replay inspection and finite exponential backoff.
Traceable processing data to evidence	**06-document-pipeline-studio**	Format routing, structural chunks, line/page traceability, deterministic extraction and document diffing.

Operating principle
A passing build is a gate, not a conclusion. The result still needs boundary tests, runtime observation and a decision that matches the evidence.

GPU KERNEL PERFORMANCE LAB

Measure the kernel. Respect the hardware boundary.

The public CUDA lab keeps reference paths, CUDA sources, CMake targets and a benchmark protocol in the same repository. The current evidence is CPU-only, so the portfolio does not convert it into a GPU performance claim.

KERNEL SURFACE

TRANSPOSE	naive global memory / tiled shared memory / padded tile
REDUCTION	atomic baseline / shared memory / warp shuffle
GEMM	naive matrix multiply / 16 x 16 tiled path / cuBLAS reference
VECTOR ADD	guarded 1D launch / size cap / CPU reference

kernels / transpose.cu

```
__shared__ float tile[32][32 + 1];
const auto col = blockIdx.x * 32 + threadIdx.x;
const auto row = blockIdx.y * 32 + threadIdx.y;
if (row < rows && col < cols)
  tile[threadIdx.y][threadIdx.x] = input[row * cols +
col];
__syncthreads();
```

RECORDED CPU REFERENCE EVIDENCE

CASE	BASELINE	CANDIDATE	RATIO
transpose 1024	7.6546 ms	3.0880 ms	**2.479x**
transpose 2048	44.9157 ms	18.8515 ms	**2.383x**
reduction 1M	1.0542 ms	0.9037 ms	**1.167x**
GEMM 256	0.6024 ms	6.4042 ms	**0.094x** regression

Measurement note
Values are from the repository's CPU fallback report. They are useful for protocol and algorithm inspection, not as CUDA throughput evidence.

SCIENTIFIC REPRODUCIBILITY KIT

Environment + benchmark + decision + source hashes + manifest.

- relative paths
- finite timing
- explicit correctness
- hash mismatch = fail

Boundary that matters: the repository records `NOT BENCHMARKED ON CURRENT HARDWARE`; the portfolio keeps that statement intact.

public source ->

AGENTIC GPU OPTIMIZER

Generated code is a hypothesis, not a verdict.

The workflow treats an Agent suggestion as a candidate patch. It validates paths, applies the diff inside a fresh workspace, runs correctness before timing, and turns benchmark evidence into an explicit decision.



REAL CASE STUDIES

CASE	CORRECTNESS	PERFORMANCE	DECISION
transpose padded tiled candidate	PASS	4.3958 -> 0.0452 ms 97.25x CPU-only	INCONCLUSIVE
reduction precision candidate	PASS	0.3335 -> 2.2744 ms 0.147x CPU-only	REJECT

The first case clears the CPU gate but cannot become a CUDA acceptance without CUDA evidence. The second is correct but regresses and is rejected.

optimizer / evaluator / decision.py

```
if not correctness_gate["passed"]:
    decision = "REJECT"
elif performance_gate["status"] ≠
"PASS":
    decision = "REJECT"
elif not
performance_gate["cuda_validated"]:
    decision = "INCONCLUSIVE"
else:
    decision = "ACCEPT"
```

The decision function encodes what the evidence is allowed to prove.

8 pytest tests passed in the extracted public copy

2 case studies re-run locally

3 decision states retained

public source ->

PROVES evaluation order, workspace isolation and explicit outcomes.

DOES NOT PROVE CUDA throughput on this host.

RAG POISONING DETECTION + TRUSTED PROVENANCE

Make the evidence chain inspectable.

The public FastAPI prototype combines source attestation, risk signals, claim extraction, conflict analysis, hybrid retrieval, evidence scoring, red-team generation and quarantine. It is presented as a prototype with a known frontend-source boundary.




```
PoisonRisk(d) = 0.25 source + 0.35 conflict
+ 0.25 similarity + 0.15 injection
```

Risk rules	prompt injection, hidden text, authority impersonation, time confusion, low-trust sources
Decision signals	source confidence, freshness, integrity, conflict penalty and claim similarity
Recovery path	risk score 80+ is quarantined in the self-purification demo

backend / services / evidence.py

```
score = relevance * 0.30 \
  + source_reliability * 0.25 \
  + document_integrity * 0.20 \
  + temporal_validity * 0.15 \
  - conflict_penalty * 0.10
```

TESTED IN EXTRACTED COPY

- 6 backend tests passed
- prompt injection + hidden text detection
- quarantined chunk exclusion
- Chinese bigram + ASCII tokenization
- evidence conflict penalty
- health and version API smoke

EXPLICIT LIMITS

- frontend source is not included in the public repository
- frontend is previewable from a built distribution
- local / trusted-LAN prototype only
- synthetic datasets and labels

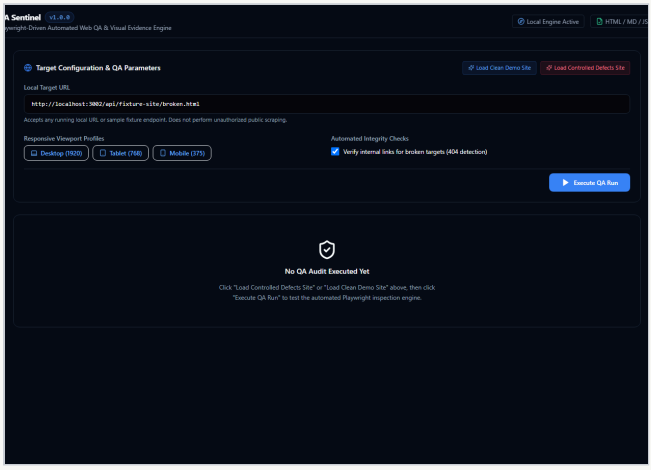
public source ->

backend + previewable frontend

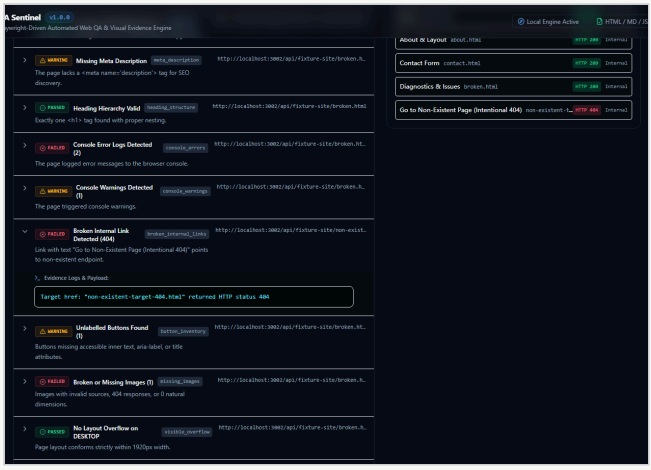
02-qa-sentinel

Find the defect, show the evidence.

QA Sentinel turns a local URL or controlled fixture into a reproducible audit. The runner observes the browser, checks page structure and assets, tests three viewports and emits HTML, Markdown and JSON evidence.



Target configuration, local fixture and viewport profiles.



Controlled defect fixture: 404 link, console error, missing image and unlabeled button.

INSPECTION SURFACE

HTTP + console + page errors
links + images + buttons + forms
title + meta + H1 + overflow

VIEWPORTS

1920 x 1080 desktop
768 x 1024 tablet
375 x 667 mobile

RE-RUN EVIDENCE

2 unit tests passed
1 E2E scenario passed
1 production build passed
controlled fixture returned
VERDICT: FAIL

Why it matters for Xpert: the project tests both clean and intentionally broken pages, then keeps the failed run inspectable instead of hiding it behind a green summary.

CLEAN RUN

baseline fixture passes

DEFECT RUN

known failures stay visible

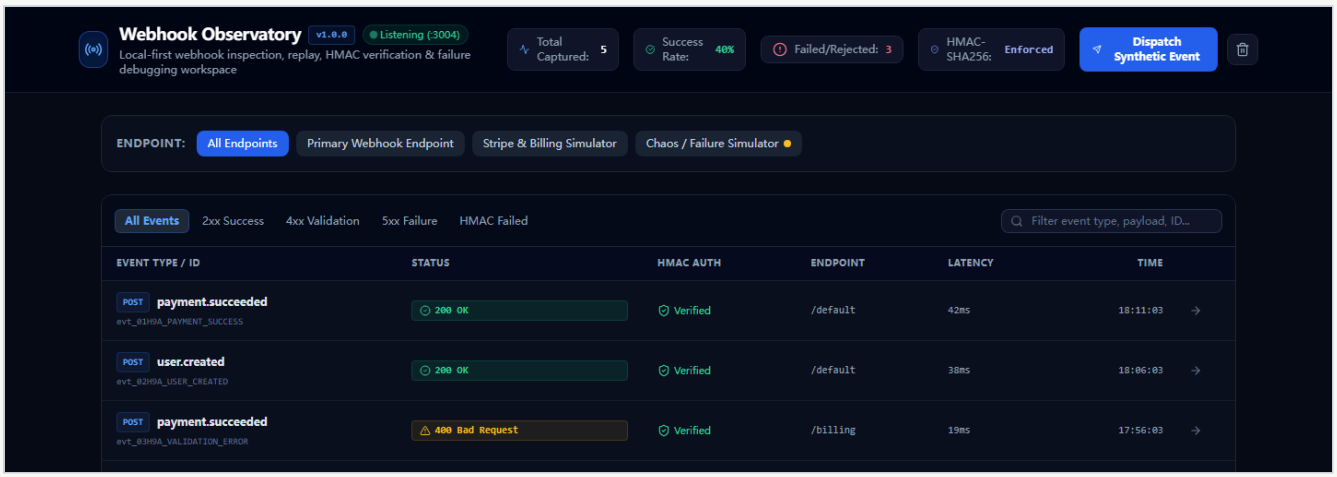
ARTIFACT

HTML / Markdown / JSON

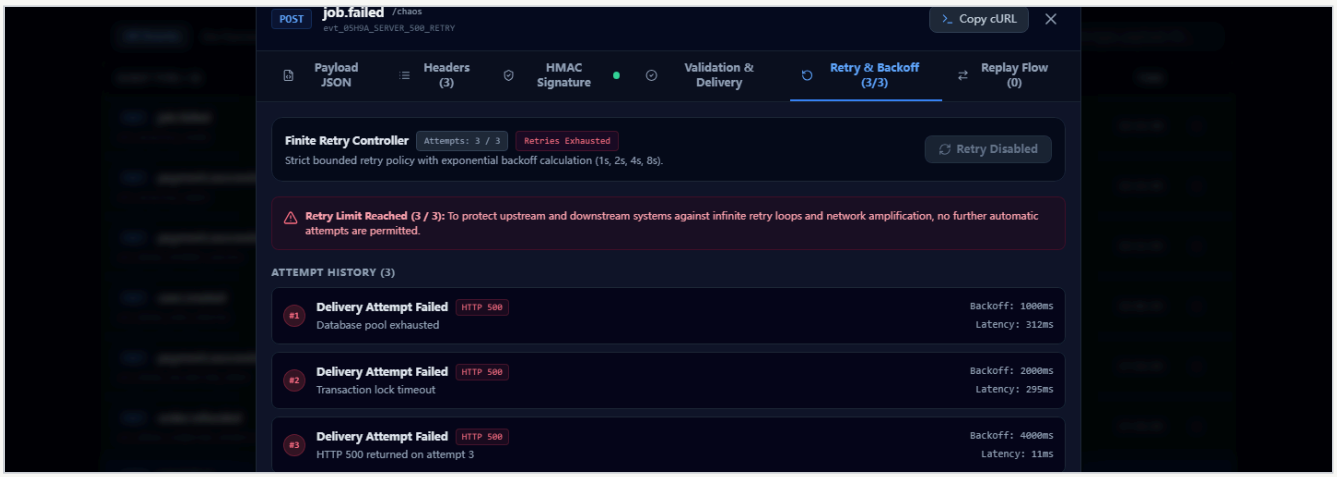
04-webhook-observatory

Make failure paths visible before production.

This local-first webhook workbench makes cryptographic verification, contract failures, replay behavior and bounded retries inspectable in one surface.



Captured event inbox with status, HMAC result, endpoint and latency.



Failure drill: three attempts, 1s / 2s / 4s backoff, retries exhausted.

SECURITY CHECKS

timingSafeEqual() for HMAC digests
300 s default timestamp tolerance
Stripe-style and GitHub-style signatures
contract validation before delivery

RESILIENCE CHECKS

3 default retries
5 absolute cap
bounded exponential backoff
server error / timeout / validation simulation

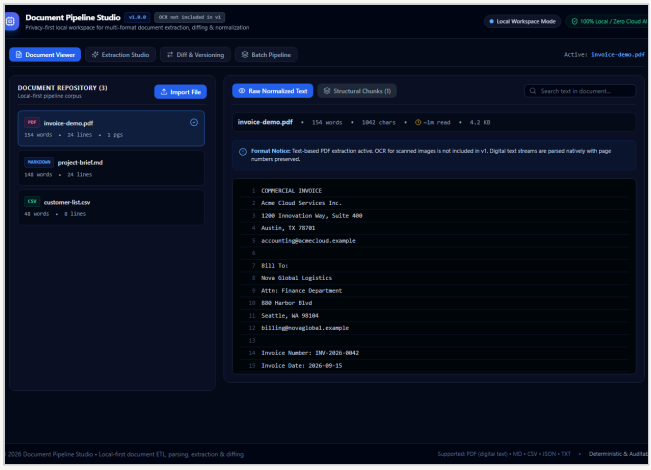
RE-RUN EVIDENCE

16 / 16 unit + contract tests
1 E2E lifecycle passed
7 routes built cleanly

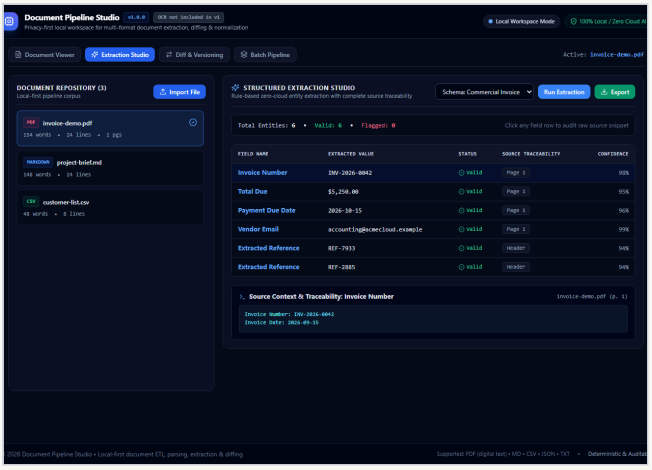
06-document-pipeline-studio

Turn raw documents into auditable structure.

Document Pipeline Studio combines a FastAPI parsing engine with a React workspace. The important detail is not only extraction: each field carries source document, page or section, line number and a literal snippet.



Multi-format repository: digital PDF, Markdown and CSV samples.



Rule-based extraction with source traceability and confidence.

PIPELINE

1. format detection
2. normalization + structural chunks
3. standard or custom schema extraction
4. line/page evidence snippet
5. diff, batch and JSON/Markdown export

RE-RUN EVIDENCE

12 backend Pytest tests passed
1 frontend E2E flow passed
frontend production build passed
sample PDF extraction validated

SCOPE BOUNDARY

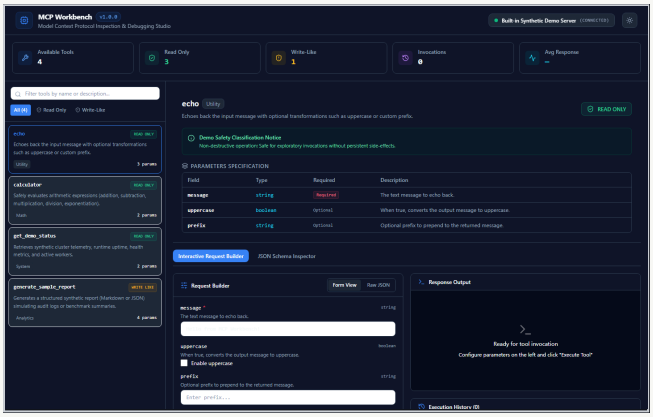
v1: digital vector text only. Scanned-image PDF OCR is excluded. Scope clarity is part of engineering quality.

Review signal: the backend tests cover format detection, PDF parsing, schema extraction, required-field failure and line diff similarity.

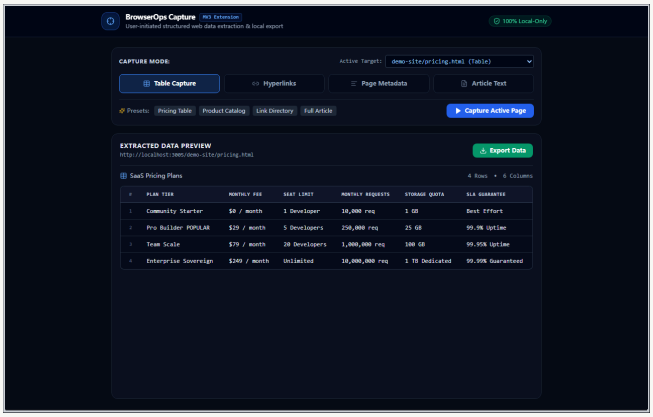
SECONDARY EVIDENCE

Range, without quantity theater.

Three additional local systems support the same evaluation profile. They are included here as compact evidence, not as separate marketing stories.



01 MCP Workbench / schema inspection, tool safety heuristic, local demo server.



05 BrowserOps Capture / MV3 activeTab extraction, table parsing, RFC 4180 export.

PROJECT	ENGINEERING SIGNAL	RE-RUN EVIDENCE	LIMIT
01-mcp-workbench	JSON Schema driven request builder, execution history and read/write-like tool classification.	14 unit tests / 2 E2E / build pass	classification is heuristic; calculator boundary noted.
05 BrowserOps	Manifest V3 minimal permission model, DOM extraction and local CSV/JSON/Markdown serialization.	10 unit tests / 1 E2E / build pass	no Canvas/WebGL or infinite-scroll extraction.
03 Data Cleanroom	Column profiling, dedupe, currency/date normalization, rule validation and before/after audit.	9 unit tests / 1 E2E / build pass	browser memory workload, about 50,000 rows.

These projects reinforce a consistent pattern: user-scoped access, deterministic transformations, explicit failure states and exportable evidence.

PORTFOLIO AUDIT

A portfolio that says what is known.

Every project page is tied to a source file, test result, screenshot or public repository. Claims that were not verified are kept out of the final pages.

EVIDENCE MAP / REPRESENTATIVE SOURCES

P03	**GPU kernel lab**	README.md / kernels/*.cu / tests / results/benchmark_results.json
P04	**Agentic GPU optimizer**	optimizer/orchestrator.py / evaluator/decision.py / experiments/*/decision.json
P05	**RAG trusted provenance**	backend/services/detector.py / consistency.py / evidence.py / rag.py / tests
P06	**QA Sentinel**	src/engine/runner.ts / tests / controlled fixture / screenshots
P07	**Webhook Observatory**	src/lib/hmac.ts / retry.ts / validator.ts / tests / release validation
P08	**Document Pipeline**	backend/parsers.py / extractor.py / diff_engine.py / tests / screenshots

PRIVACY AUDIT

PASS	no names, phone numbers or email accounts
PASS	no tokens, keys, cookies or SSH material
PASS	no browser profile or login state
PASS	no private repo URL or local path in final PDF
PASS	synthetic samples only in displayed screenshots
PASS	original projects remained read-only

NOT CLAIMED

No game or Minecraft Mod directory was present in the authorized local library scan. No GPU result is presented as a GPU result. No user, star, fork or download metric is used.

Selection rule

Authenticity > engineering completeness > verifiability > technical depth > visual polish > project count.

PUBLIC SOURCES

Agentic GPU Optimizer

GPU Kernel Performance Lab

RAG Trusted Provenance